GSD RESEARCH MISSION PACKAGE

Spatial Awareness
The Chorus Protocol

Paula Chipset — Agent Environment Intelligence

Passive Sensing, Threat Assessment, and Coordinated Signaling for Agent Systems

Vision → Research → Mission

Full Pipeline Execution

Date: March 8, 2026 | Status: Ready for GSD Orchestrator

Activation Profile: Squadron | Estimated: 14–18 context windows across 4–5 sessions

Contents

1 Spatial Awareness — Vision Guide **4**
1.1 Vision . . . 4
1.2 Problem Statement . . . 7
1.3 Core Concept . . . 8
1.3.1 The Frog Protocol — Threat Lifecycle . . . 8
1.3.2 Three Communication Tiers . . . 8
1.3.3 Passive Environmental Sensing . . . 9
1.3.4 Module Architecture . . . 9
1.4 Research Modules . . . 10
1.4.1 M1: Passive Environmental Sensor . . . 10
1.4.2 M2: Threat Detection Engine . . . 10
1.4.3 M3: Environmental Geometry Mapper . . . 10
1.4.4 M4: Graduated Response Controller . . . 11
1.4.5 M5: Multi-Tier Communication Bus . . . 11
1.4.6 M6: Chorus Coordination Protocol . . . 11
1.5 Chipset Configuration . . . 12
1.6 Scope Boundaries . . . 12
1.6.1 In Scope (v1.0) . . . 13
1.6.2 Out of Scope . . . 14
1.7 Success Criteria . . . 14
1.8 Relationship to Other Documents . . . 15
1.9 The Through-Line . . . 15

2 Spatial Awareness — Research Reference **17**
2.1 How to Use This Document . . . 17
2.2 Distributed Coordination Without Consensus . . . 17
2.2.1 The Frog Chorus as Distributed System . . . 17
2.2.2 Broadcast Protocols for Instant Coordination . . . 18
2.3 Signal Detection in Noisy Environments . . . 18
2.3.1 The Cocktail Party Problem and “Order Up” . . . 18
2.3.2 Matched Filtering for Signal Extraction . . . 19
2.4 Passive Sensing and Ambient Intelligence . . . 19
2.4.1 Echolocation Without Emission . . . 19
2.4.2 Computational Ambient Signals . . . 19
2.5 Physical Sensor Integration via USB . . . 20
2.5.1 USB Device Classes . . . 21
2.5.2 Connection Architecture . . . 22
2.5.3 Reference Device: Samsung Galaxy S25 Ultra . . . 22
2.6 Sensor-Driven Output Synthesis . . . 24
2.6.1 Audio Synthesis . . . 24
2.6.2 LED Control . . . 24
2.6.3 Laser Projection . . . 25
2.6.4 Mapping Engine . . . 25
2.7 Safety and Sensitivity Considerations . . . 26

2.8 Source Bibliography . . . 26

3 Milestone Specification **28**
3.1 Mission Objective . . . 28
3.2 Architecture Overview . . . 28
3.3 System Layers . . . 28
3.4 Deliverables . . . 29
3.5 Component Breakdown . . . 30
3.6 Model Assignment Rationale . . . 30
3.7 Wave Execution Plan . . . 30
3.7.1 Wave Summary . . . 31
3.7.2 Wave 0: Foundation . . . 31
3.7.3 Wave 1: Parallel Tracks . . . 31
3.7.4 Wave 2: Integration . . . 33
3.7.5 Wave 3: Verification + Publication . . . 34
3.7.6 Cache Optimization Strategy . . . 34
3.7.7 Token Budget . . . 34
3.7.8 Dependency Graph . . . 35
3.8 Test Plan . . . 35
3.8.1 Test Categories . . . 35
3.8.2 Safety-Critical Tests . . . 35
3.8.3 Core Functionality Tests . . . 36
3.8.4 Integration Tests . . . 37
3.8.5 Edge Case Tests . . . 37
3.8.6 Verification Matrix . . . 38
3.9 Mission Crew Manifest . . . 38
3.10 Execution Summary . . . 39

STAGE 1 — VISION DOCUMENT

1 Spatial Awareness — Vision Guide

Date: March 8, 2026

Status: Initial Vision / Conversation-Harvested

Depends on: deep-audio-analyzer-mission (Paula Layer 1), gsd-chipset-architecture-vision.md, gsd-skill-creator-analysis.md, gsd-mission-crew-manifest.md

Context: The second release of the Paula audio chipset. Where the Deep Audio Analyzer teaches Paula to hear, Spatial Awareness teaches Paula to *be aware* — to sense environments, detect threats, coordinate teams, and communicate through acoustic channels. This is the frog's perspective.

1.1 Vision

Consider the frog.

Not the frog as data — that was the first release, the Deep Audio Analyzer, where we sat at the edge of the pond and decoded their world from the outside. Now consider the frog as *agent*. The frog is not passively producing data. The frog is running a survival protocol. It has objectives (attract mates, defend territory), it has threats (predators, competitors, weather), it has teammates (the chorus), and it has a communication system that operates under constraints no software engineer would willingly design: the channel is shared with every predator in earshot, the environment distorts every signal, and silence is the only safe default.

Watch what happens when the fox arrives at the pond. The frogs go silent. Not because a central controller sent a `PAUSE` command. Each frog independently detected a change in its environment — a footstep vibration, a shadow, a pressure wave — and independently decided to stop calling. This is distributed threat detection with zero coordination overhead. No message bus, no leader election, no consensus protocol. Just twenty-five seconds of shared silence, and then one brave soul testing the water with a tentative 10-millisecond chirp.

That chirp is not a broadcast. It is a *probe*. The frog is sacrificing its own safety to generate information for the group. If the fox moves, the frog dies and the others learn. If the fox stays still, the frog survives and the others learn differently. After enough probes without consequence, the chorus restarts — cautiously, then fully. And here is the remarkable thing: when the fox eventually stands up to leave, the chorus does not stop. The frogs have reclassified the fox from "threat" to "furniture." The spatial model has been updated. The threat object has been tracked, assessed, and dismissed.

This is Spatial Awareness: the ability for agents in a system to sense their operating environment, detect anomalies and threats, assess danger through graduated probing, coordinate responses without centralized control, communicate through channels appropriate to the threat level, and maintain a continuously-updated model of

who and what is around them.

The applications are immediate and practical:

A person in a dark unfamiliar space holds up their phone. The microphone listens. Sound reflections off walls, ceiling, furniture paint a geometric picture of the room. The phone vibrates patterns against the palm — *wall 2 meters ahead, open space to the left, low ceiling* — without emitting any light or sound that would reveal the person's position. The phone is a bat. The vibrations are its sonar display.

A kitchen in full dinner rush. The line cook cannot see the servers. The servers cannot see the kitchen. But "ORDER UP!" cuts through the chaos — a trained signal pattern that every server's brain has learned to extract from the noise. The signal does not need to be louder than the noise. It needs to be *different* from the noise. The right spectral signature at the right moment reaches every agent in the system simultaneously.

A team of GSD agents working in parallel. The supervisor agent needs to pause all work immediately — a critical dependency has changed, a safety boundary has been crossed, a human has called for attention. The pause signal must reach every agent regardless of what they are currently doing, regardless of what tools they are using, regardless of how deep in their task they are. This is the chorus going silent. One signal. Every agent hears it. Every agent stops. No acknowledgment required. No round-trip. No consensus. Silence is instant.

The phone by the pond was a proof of concept. The real application space is everywhere a sensor can reach — and everyone already carries a phone full of sensors.

Walking through the forest with a USB microphone. The laptop is in the pack, the mic clipped to the strap. Birdsong fills the canopy — Swainson's thrush, Pacific wren, varied thrush. The system identifies each species from its spectral signature and lists them as icons on the page, a living field guide that populates itself from the forest's own broadcast. No button press, no query. The forest announces itself through the microphone and the page reflects what it hears. Passive sensing applied to biodiversity — the AVI research atlas fed by real-time audio from a USB device.

DJ booth at the burn, syncing to neighboring camps. Three sound camps within earshot. Your set needs to complement, not clash. A USB mic pointed outward captures the ambient beat — 128 BPM from the camp to the east, 126 from the south. The system extracts tempo via onset detection and autocorrelation, feeds it to your DAW's sync. Your music locks to the playa's pulse. This is the frog chorus in reverse: instead of synchronizing to stop, you synchronize to *play together*. Tempo sync emerging from local sensing, no coordinator, no shared clock.

A build agent that listens while it works. The agent is compiling code, running tests, writing files. Simultaneously, a USB microphone feeds ambient audio into the passive sensor. The agent hears the environment change — a door opens, background noise shifts. The spatial model updates without the agent pausing its primary task. It simply *knows* the environment changed, the way a frog keeps calling while tracking every shadow and vibration around the pond.

A machinist operating a lathe, and it is loud. The shop floor is 90 dB. Voice

commands are useless. You cannot hear things correctly. But the USB mic on the CNC controller hears the tool chatter, bearing frequency, spindle harmonics. A shift in cutting sound means tool wear or workpiece movement. The system detects the anomaly through matched filtering — the same technique that extracts "ORDER UP!" from kitchen chaos — and sends a haptic alert to the operator's wrist. The signal cuts through noise not by being louder, but by being *different*.

Driving in a car. Road noise, engine, wind — broadband interference. But a mic still hears the siren approaching (Doppler-shifted, rising pitch), the rhythmic thump of a flat tire, the acoustic change when a window cracks open. Hostile to human hearing but structured enough for matched-filter extraction.

A phone is already a sensor array. Every phone carries a microphone, accelerometer, gyroscope, magnetometer, barometer, GPS, WiFi radio, Bluetooth radio, and camera. Connected over USB to a laptop, or running as a standalone node, it is a complete spatial awareness platform. No special hardware. No purchase required. The sensors are already in your pocket.

Phones as mesh nodes. Multiple phones in proximity form a mesh network — WiFi Direct, Bluetooth, or ad-hoc WiFi. Each phone senses its local environment (audio, motion, RF signal strength) and shares observations with nearby nodes. Link the mesh to a WiFi access point and the local sensing network bridges to the broader system. This is DIY infrastructure: the mesh you build with the phones around you *is* the sensing network. Building the mesh is participating. Each node is a frog in the chorus — sensing independently, coordinating through proximity, no central controller required.

Radios as broadcast channels. Handheld radios, LoRa modules, Meshtastic devices, RTL-SDR dongles — all accessible over USB. A radio is a long-range broadcast channel: the chorus's call reaching beyond the immediate pond. LoRa reaches 10+ kilometers on the playa. An RTL-SDR dongle is pure passive sensing — listening to the RF environment without transmitting, the electronic equivalent of the frog feeling vibrations through the ground.

An art installation that breathes with the crowd. Accelerometers on the playa detect footsteps, density, flow. Microphones capture ambient BPM from neighboring sound camps. The sensor data feeds a synthesis engine that generates a drone — low, warm, shifting with the crowd's energy. WS2812B LED strips woven through the structure pulse in phase with the drone, colors drifting from deep amber to electric blue as density rises. The installation *responds* to the humans around it without anyone pressing a button. The frogs do not just listen — they sing back. Sensing is input. Synthesis is output. The chorus is both.

Laser skywriting from the spatial model. A USB-connected ILDA galvo scanner projects geometric patterns onto fog, dust, or structure. The patterns are not preprogrammed — they are derived from live sensor data. The geometry mapper's model of the environment becomes visible: walls of light showing constraint boundaries, pulsing nodes where agents are active, sweeping beams that track threat vectors. On the playa, the laser draws the shape of the mesh network in real time — each phone node a projected point, connections traced as lines of light. The spatial model made literal. The frog's awareness of its pond, painted in coherent light across the

desert sky.

DMX light rigs driven by the Frog Protocol. When the chorus goes silent — threat detected — the LED fixtures dim to deep red, pulsing slowly. During ASSESS, amber scanning patterns sweep the space. PROBE sends a single sharp beam outward. CLASSIFY resolves to green (safe) or strobing red (danger). RESUME brings the full spectrum back, gradually, the way the frogs restart their calling. The Frog Protocol's five phases become a light show that anyone in the space can read without understanding the protocol. The light *is* the protocol, made visible.

There are many more use cases. Every environment where humans or machines operate has a signature — acoustic, electromagnetic, vibrational — and that signature changes when the environment changes. The question is not "what can we connect to" but "what *can't* we connect to." The answer: not much. USB is the bridge. The phone is already the sensor. The mesh is the chorus. And now the chorus sings back — in sound, in light, in laser geometry projected onto whatever surface the world provides.

1.2 Problem Statement

1. **Agents operate blind to their environment.** Current GSD agents know what task they are executing and what tools they have access to, but they have no awareness of the broader operational context — what other agents are doing, what the resource landscape looks like, whether external conditions have changed, or whether the environment itself has shifted. An agent in a pipeline has no peripheral vision.

2. **Threat detection is centralized and slow.** When something goes wrong in a GSD mission — a dependency breaks, a safety boundary is approached, a budget ceiling is hit — the detection and response path runs through the Flight Director. This is the equivalent of every frog in the pond waiting for a designated lookout to announce danger. It works, but it is slow, it has a single point of failure, and it does not scale.

3. **There is no graduated threat response.** Current systems are binary: everything is fine, or BLOCK. There is no equivalent of the frog's twenty-five-second assessment period, no tentative probe, no graduated re-engagement. An agent that encounters an anomaly either ignores it or escalates to a full stop. The middle ground — pause, assess, probe, resume cautiously — does not exist.

4. **Communication channels do not adapt to context.** All agent communication flows through the same mechanism regardless of urgency, sensitivity, or audience. There is no distinction between a broadcast that must reach every agent instantly ("PAUSE ALL"), a directed message to a specific agent ("check this dependency"), and a covert signal that should not be visible to other processes (haptic feedback to a human). The "order up" call and the whispered correction use the same channel at the same volume.

5. **Agents cannot sense the shape of their operating space.** In the physical world, sound reflections tell you whether you are in a closet or a cathedral. In the computational world, agents have no equivalent sense. An agent does not

know whether it is operating in a resource-rich or resource-scarce environment, whether the context window is nearly full or mostly empty, whether other agents are idle or saturated, or whether the "walls" of its operating constraints are close or far. It has no spatial model of its own operational geometry.

6. **No passive sensing capability exists.** Every information-gathering action in the current system requires active queries — tool calls, file reads, API requests. There is no mechanism for an agent to *listen* to ambient signals in its environment without generating observable side effects. In a covert or resource-constrained context, every query is a footstep that might spook the frogs.

1.3 Core Concept

Silence → Detect → Assess → Probe → Classify → Adapt

The Spatial Awareness system implements the frog's threat lifecycle as a general-purpose agent coordination protocol. It provides three communication tiers (covert, directed, broadcast), a passive environmental sensing capability, a graduated threat response system, and a continuously-updated spatial model of the agent's operating environment.

1.3.1 The Frog Protocol — Threat Lifecycle

```
THE FROG PROTOCOL — Graduated Threat Response
BASELINE: Agents working (chorus calling) | [ANOMALY DETECTED] —
environmental change sensed | v PHASE 1: SILENCE (immediate, distributed)
- All agents pause current work - No acknowledgment required - State
preserved for resume - Duration: configurable (default 5s equivalent)
| v PHASE 2: ASSESS (passive observation) - Designated scout agent
characterizes the anomaly - Other agents remain silent - Environment
monitored for additional signals - No active probing yet | v PHASE 3:
PROBE (graduated testing) - Scout agent makes minimal test action - If
probe succeeds without adverse reaction: escalate - If probe fails: extend
silence, escalate threat level - Multiple probes with increasing scope |
v PHASE 4: CLASSIFY (threat model update) - Anomaly classified: THREAT /
NEUTRAL / OPPORTUNITY - Spatial model updated with new object - If THREAT:
maintain heightened awareness - If NEUTRAL: resume with object tracked -
If OPPORTUNITY: adapt strategy to exploit | v PHASE 5: RESUME (graduated
re-engagement) - Agents resume in priority order (not all at once) -
Threat object remains in spatial model - Awareness level remains elevated
- Full chorus restored only after sustained safety
```

1.3.2 Three Communication Tiers

```
COMMUNICATION ARCHITECTURE — The Three Channels
TIER 1: COVERT (Haptic / Private) Direction: Point-to-point Visibility:
Sender and receiver only Analogy: Phone vibrating in your pocket Use
cases: Human notification without broadcast Agent-to-agent private signal
Stealth mode feedback GSD impl: CAPCOM -> human via dashboard notification
Agent -> agent via direct message queue System -> human via non-visual
feedback
TIER 2: DIRECTED (Whisper / Targeted) Direction: One-to-one or one-to-few
```

```
Visibility: Named recipients only Analogy: Speaking to the person next
to you Use cases: Task delegation without team disruption Correction or
redirection of specific agent Specialist consultation GSD impl: Agent
addressing by name in message bus Scoped skill activation (context: fork)
TOPO restructuring specific agents
TIER 3: BROADCAST (Order Up / All-Hands) Direction: One-to-all Visibility:
Every agent in the system Analogy: "ORDER UP!" / chorus going silent Use
cases: Emergency pause (safety boundary) Phase transition announcement
Resource alert (budget ceiling) Go/No-Go signal at wave boundary GSD impl:
FLIGHT directive via mission bus Interrupt-level signal (pre-empts current
task) File-system signal (.planning/PAUSE)
```

1.3.3 Passive Environmental Sensing

```
PASSIVE SENSING — The Bat Phone Model
The problem: How does an agent know its environment without asking
questions that change the environment?
Physical analogy: Phone microphone in a dark room. Ambient sounds + their
reflections = room geometry. Output via haptic (vibration) = no light, no
sound emitted. The phone "sees" by listening.
Computational analogy: Agent reads ambient signals without tool calls:
- Context window fill level (how close are the walls?) - Token budget
remaining (how much fuel is left?) - Other agents' output rate (who's
busy, who's idle?) - File system timestamps (what changed recently?) -
Error rate trends (is the environment degrading?) - Session age (how much
time has passed?)
These are the "echoes" — information already present in the environment
that an agent can sense without generating new queries. The spatial model
is built from ambient data.
SPATIAL MODEL OUTPUT: +---------------------------------------------------+
| AGENT: executor-a | | POSITION: Wave 1, Track A, Step 3 of 5 | |
ENVIRONMENT: | | Context fill: 62| Budget remain: 34| Peer agents: 3
active, 1 idle, 1 blocked | | Threat level: NOMINAL (no anomalies) |
| Wall distance: 38| Last anomaly: 12 min ago (resolved: dep check) |
+---------------------------------------------------+
```

1.3.4 Module Architecture

```
MODULE DECOMPOSITION — SPATIAL AWARENESS
M1: Passive Environmental Sensor Context probes | Resource monitors |
Ambient signals No tool calls | No side effects | Pure observation | M2:
Threat Detection Engine Anomaly detection | Threshold monitoring | Pattern
breaks Distributed sensing (every agent participates) | M3: Environmental
Geometry Mapper Spatial model construction | Constraint mapping "Where
are the walls?" | "How much space do I have?" | M4: Graduated Response
Controller The Frog Protocol | Phase management | Probe dispatch Silence
-> Assess -> Probe -> Classify -> Resume | M5: Multi-Tier Communication
Bus Covert channel (haptic) | Directed channel (whisper) Broadcast channel
(order up) | Signal filtering | M6: Chorus Coordination Protocol Agent
sync | Distributed pause/resume | No leader required State preservation |
Graduated re-engagement
```

1.4 Research Modules

1.4.1 M1: Passive Environmental Sensor

The ambient intelligence layer. This module enables agents to sense their operating environment without generating queries, tool calls, or any observable side effects. It reads "echoes" — information already present in the computational environment: context window utilization, token budget state, filesystem timestamps, sibling agent output rates, error frequencies, and session timing. From these ambient signals, it constructs a spatial model of the agent's operational geometry — how close are the resource walls, how crowded is the workspace, how healthy is the environment. This is the bat's sonar: build a map from what bounces back, without emitting a signal.

The physical application is direct and universal. A USB microphone captures ambient sound — birdsong in a forest, tool chatter on a shop floor, road noise in a car, music from neighboring camps. A phone connected over USB contributes its full sensor array: microphone, accelerometer, gyroscope, magnetometer, barometer, GPS, WiFi RSSI, Bluetooth proximity. An ESP32 over USB serial feeds temperature, humidity, motion, light. An RTL-SDR dongle listens to the RF spectrum without transmitting. Every USB device is a sensory organ; the Passive Sensor reads them all through the same interface contract (`sensor-interface.ts`), whether the signal source is a computational ambient signal or a physical device plugged into a USB port. Where there are uplinks — WiFi, cellular, mesh radio — the sensing extends beyond the local machine. Phones form mesh networks via WiFi Direct or Bluetooth, each node sensing independently and sharing observations through proximity. The mesh bridges to the broader system wherever an uplink exists. The sensor network grows organically: every phone that joins is another frog in the chorus.

1.4.2 M2: Threat Detection Engine

The anomaly detection layer. Every agent in the system participates in threat detection — this is not centralized surveillance but distributed awareness, like every frog independently noticing the fox's footstep. The module monitors for environmental changes that exceed configurable thresholds: sudden token consumption spikes, dependency failures, safety boundary approaches, unexpected agent silence (an agent that should be producing output but isn't), or external signals from SENTINEL/upstream intelligence.

The detection is probabilistic, not binary. An anomaly has a threat level that escalates with persistence and correlation. A single blip might be noise. Two correlated blips from different agents are a pattern. Three synchronized anomalies trigger the Frog Protocol.

1.4.3 M3: Environmental Geometry Mapper

The spatial model layer. This module answers the question every frog knows instinctively: "What shape is my pond?" For computational agents, the "pond" has walls made of context window limits, token budgets, rate limits, file system boundaries, and time constraints. The geometry mapper constructs and continuously updates a model of these boundaries, giving each agent awareness of how much room it has to work, where the constraints are tightest, and how the space is changing over time.

For physical applications, this module extends the Deep Audio Analyzer's reflection

detection to active geometry reconstruction. Using the comb filtering and RT60 techniques from Release 1, it can map room dimensions from ambient sound alone — enabling the "phone in a dark room" scenario where a device builds a spatial model from microphone data and communicates the geometry through non-visual, non-audible channels (haptic patterns).

1.4.4 M4: Graduated Response Controller

The Frog Protocol implementation. When the Threat Detection Engine signals an anomaly, this module manages the five-phase graduated response: SILENCE (immediate distributed pause), ASSESS (passive characterization), PROBE (minimal test actions), CLASSIFY (threat model update), and RESUME (graduated re-engagement). The controller does *not* centralize decision-making. Each phase has clear entry and exit criteria that agents can evaluate independently. The controller provides the protocol; the agents execute it autonomously.

The critical insight from the frog pond is that resumption is graduated, not binary. The first frog to call again after a silence is the probe — it is taking a calculated risk to generate information for the group. The system must support this pattern: a designated or volunteer scout agent re-engages first while others remain paused, and the all-clear propagates only if the scout survives.

1.4.5 M5: Multi-Tier Communication Bus

The three-channel architecture. This module implements the covert, directed, and broadcast communication tiers that mirror the full range of biological signaling: the frog's body posture (visible only to nearby frogs), its directed croak (aimed at a specific rival), and the chorus itself (audible to every entity in the environment).

The "order up" pattern is the critical broadcast use case. In a noisy environment (agents producing output, tools running, files being written), a broadcast signal must cut through the chaos to reach every agent instantly. The signal achieves this not by being louder than the noise, but by being *spectrally distinct* — a pattern that stands out against the background, the way a trained server's brain extracts "ORDER UP!" from kitchen chaos. In the GSD context, this means a signal encoding that is orthogonal to normal agent communication, so it cannot be confused with regular traffic regardless of system load.

For physical applications, this maps to audio signaling in loud environments — using frequency, timing, or modulation patterns that are engineered to be extractable from ambient noise. A factory floor, a concert venue, a crowded restaurant: environments where traditional voice communication fails, but structured acoustic signals can succeed.

1.4.6 M6: Chorus Coordination Protocol

The emergent synchronization layer. The frog chorus starts, synchronizes, and stops without a conductor. Individual frogs follow local rules (call when neighbors call, stop when danger is sensed, match the tempo of nearby callers) and global coherence emerges from local behavior. This module implements the same principle for agent teams: coordinated behavior without centralized control.

The key operations are: distributed pause (every agent stops independently upon

sensing the broadcast signal, with no round-trip acknowledgment needed), state preservation (each agent snapshots its current state for seamless resume), graduated resume (scout-first re-engagement), and tempo synchronization (agents adjusting their work rate to match the team's rhythm, preventing individual agents from racing ahead or falling behind).

This module connects directly to TOPO in the crew manifest. TOPO manages team topology; the Chorus Coordination Protocol gives TOPO a mechanism for real-time synchronization that does not require message-passing overhead. It is the difference between a conductor stopping an orchestra (centralized, slow) and a flock of starlings turning simultaneously (distributed, instant).

1.5 Chipset Configuration

```
chipset.yaml — Spatial Awareness (Paula Layer 2) chipset: name:
"spatial-awareness" version: "1.0.0" description: "Agent environmental
sensing, threat response, and coordinated signaling" category:
"audio-intelligence" position: "paula" derived_from : "deep — audio —
analyzer"InheritsRelease1
skills: required: - name: "passive-sensor" context: "always"  Ambient —
no activation needed - name: "threat-detector" context: "always"  Always
listening - name: "chorus-protocol" context: "always"  Coordination is
always active
domain: - name: "geometry-mapper" context: "fork" triggers: ["space",
"room", "environment", "boundaries"] - name: "frog-protocol" context:
"fork" triggers: ["threat", "anomaly", "pause", "danger", "alert"] - name:
"signal-bus" context: "fork" triggers: ["broadcast", "signal", "order up",
"all-hands"]
agents: topology: "swarm-with-sentinel" agents: - name: "sentinel"
role: "Passive environmental sensing, ambient monitoring" model: "haiku"
Lightweight, always-on - name: "watchdog" role: "Threat detection, anomaly
correlation" model: "sonnet" - name: "cartographer" role: "Environmental
geometry, constraint mapping" model: "sonnet" - name: "conductor" role:
"Frog Protocol phase management" model: "opus"  Judgment-heavy - name:
"herald" role: "Multi-tier signal dispatch and filtering" model: "sonnet"
- name: "chorus-master" role: "Distributed sync, tempo coordination"
model: "sonnet"
communication: tiers: covert: mechanism: "direct-queue" visibility:
"point-to-point" latency: "best-effort" directed: mechanism:
"named-address" visibility: "named-recipients" latency: "normal"
broadcast: mechanism: "interrupt-signal" visibility: "all-agents" latency:
"immediate" encoding: "orthogonal-pattern"
pause_signal : file : ".planning/CHORUS_PAUSE"effect :
"allagentssnapshotstateandhalt"resume : ".planning/CHORUS_RESUME"
token_budget : ceiling : "3passive_sensor : " 500tokens/check"threat_response :
" 5Ktokensperincident"broadcast_signal : " 100tokens"
media: reference_audio : −path : "media/audio/20260209_194520.wav"description :
"ReferencerecordingdemonstratingallSpatialAwarenessconcepts :
threatarrival(0.5s), silenceperiod(0 − 25s), graduatedprobing(18 − 25s), chorusrecruitment(25 −
90s), fullcoordination(90 − 139s), andthreatreclassification(foxacceptedasnon − threat)."
```

1.6 Scope Boundaries

1.6.1 In Scope (v1.0)

- Passive environmental sensing from ambient computational signals (context fill, budget, timing, error rates)
- Distributed threat detection with configurable thresholds and multi-agent correlation
- The Frog Protocol: five-phase graduated threat response (Silence, Assess, Probe, Classify, Resume)
- Environmental geometry mapping for computational constraints (context walls, budget ceilings)
- Three-tier communication bus: covert (point-to-point), directed (named), broadcast (interrupt)
- Broadcast signal encoding designed for extraction from noisy agent environments
- Distributed pause/resume with state preservation and graduated re-engagement
- Integration with Deep Audio Analyzer (Release 1) for physical spatial sensing applications
- File-system-based signal protocol (`.planning/CHORUS_PAUSE`, `.planning/CHORUS_RESUME`)
- Integration with TOPO, FLIGHT, and CAPCOM crew roles
- Reference recording serves as conceptual test case for all protocols
- USB physical sensor interface: unified `sensor-interface.ts` contract supporting USB audio (microphone), USB serial (ESP32/Arduino), and USB bulk (RTL-SDR) device classes
- Phone-as-sensor-node: USB-tethered phone contributing mic, accelerometer, gyroscope, magnetometer, barometer, GPS, WiFi/BT RSSI to the passive sensor layer
- Mesh networking foundation: phone-to-phone sensing via WiFi Direct / Bluetooth, bridging to WiFi uplinks where available
- Physical application scenarios: birdsong species identification, BPM/tempo detection, industrial acoustic monitoring, automotive noise extraction
- Sensor-driven audio synthesis: real-time generative sound from spatial model data via USB audio output (cpal)
- LED control: WS2812B/NeoPixel strips via ESP32 serial, DMX-512 fixtures via USB-DMX adapter
- Laser projection: ILDA galvo scanner control via USB for spatial model visualization and image projection (with mandatory safety interlock)
- Output mapping engine: declarative sensor-to-output bindings, hot-reloadable configuration, multi-source blending

1.6.2 Out of Scope

- Haptic output via phone vibration motor (v1.1 — v1.0 defines interface only; gamepad rumble is in scope via HID)
- Multi-hop mesh routing beyond direct WiFi/BT proximity (requires dedicated mesh protocol — v2)
- Autonomous threat response without human approval for safety-critical situations
- Predictive threat modeling (threat anticipation from historical patterns — v2 feature)
- Cross-mission spatial awareness (agents in different missions sensing each other)
- Custom phone app for standalone sensor node (v1.0 uses USB tether; standalone app is v1.1)
- Cellular/LTE uplink bridging (v1.0 scope is local WiFi mesh; WAN bridging is v2)

1.7 Success Criteria

1. **Passive sensor detects context window utilization** within 2% accuracy without making tool calls, by reading available ambient signals only.
2. **Distributed threat detection correctly identifies anomalies** across a test corpus of 15 simulated scenarios (dependency failure, budget spike, agent silence, safety approach) with $\geq$85% true positive rate and $\leq$10% false positive rate.
3. **The Frog Protocol completes a full five-phase cycle** (Silence through Resume) within 30 seconds for a simulated anomaly, with all agents correctly preserving and restoring state.
4. **Broadcast signal reaches all active agents within 1 second** of dispatch, verified by timestamp comparison across agent acknowledgment logs.
5. **Broadcast signal is correctly extracted from simulated noise** in a test with 5 concurrent agents producing normal output. Zero missed broadcasts across 50 test signals.
6. **Covert channel delivers messages invisible to non-recipient agents.** Verification: non-recipient agents' logs contain zero references to covert messages across a test run.
7. **Environmental geometry mapper correctly identifies resource constraints** (context window headroom, budget remaining, rate limit proximity) within 5% accuracy for each dimension.
8. **Graduated resume correctly sequences agent re-engagement.** Scout agent re-engages first; remaining agents resume only after scout success signal. Verified across 10 simulated cycles.
9. **State preservation across pause/resume loses zero work.** Agent output before pause and after resume are continuous — no duplicate work, no dropped work, no context corruption.

10. **The chorus coordination protocol achieves distributed pause** across 5 simulated agents within 500ms, without requiring a centralized controller or round-trip acknowledgment.
11. **Integration with Deep Audio Analyzer** allows physical spatial sensing output (room geometry from ambient audio) to feed into the environmental geometry mapper's spatial model format.
12. **Total awareness overhead stays below 3% of context window** during normal operation (non-threat conditions). Threat response phases may temporarily exceed this ceiling.

1.8 Relationship to Other Documents

Document	Relationship
deep-audio-analyzer-mission	Direct predecessor. Release 1 of the Paula chipset. Spatial Awareness inherits the DSP toolkit, spatial reasoning engine, and reflection detection capabilities. Physical sensing applications (phone-in-dark-room) use Release 1's analysis engines.
gsd-mission-crew-manifest	The Chorus Coordination Protocol provides TOPO with real-time synchronization. Broadcast signals flow through FLIGHT. Covert channels extend CAPCOM. Threat detection augments SURGEON.
gsd-chipset-architecture-vision	Spatial Awareness occupies the Paula position alongside Release 1. The passive sensor reads chipset-level telemetry (token budget, context fill) that the chipset architecture exposes.
gsd-staging-layer-vision	The Frog Protocol's ASSESS phase parallels the staging layer's intake flow. Both implement "pause and think before acting." Threat signals from Spatial Awareness can trigger staging layer gates.
gsd-upstream-intelligence-pack	SENTINEL agent in UIP and the sentinel agent here share the same design pattern: always-on, lightweight monitoring of ambient signals. UIP watches upstream channels; Spatial Awareness watches the local environment.
gsd-skill-creator-analysis	Chorus coordination patterns (which pause/resume sequences work well, which threat classifications are accurate) feed into skill-creator's observation pipeline for learning and refinement.

1.9 The Through-Line

A signal molecule is a letter written in the dark.

The Hundred Voices — the proof that runs through *The Space Between* — describes cells that cannot see each other, cannot touch each other, cannot know whether the cell they are writing to even exists. And yet the cell sends. Insulin released into the bloodstream has no address, no envelope, no stamp. It is a sermon delivered into the

dark, and the congregation, if there is a congregation, is scattered across an ocean of tissue.

The frog's chirp into the silence after the fox arrives is the same letter. The "ORDER UP!" from a kitchen the servers cannot see is the same letter. The broadcast pause signal that reaches every agent in a swarm without waiting for acknowledgment is the same letter. They all solve the same problem: how does a bounded self communicate across the space between, when the space is dark, the channel is noisy, and the recipient may or may not be listening?

The Amiga's Paula chip solved it with DMA channels — four audio streams that moved data to exactly the right place at exactly the right time without CPU intervention. No negotiation, no handshake, no overhead. The data moved because the architecture knew where it needed to go. Spatial Awareness solves it the same way: the broadcast signal reaches every agent because the architecture is wired for it, not because each agent polled for it. The pause file appears in `.planning/` and every agent senses it the way every frog senses the footstep — not because they were told to listen, but because they cannot help but hear.

The space between agents is where coordination lives. And coordination, like the frog chorus, is not conducted. It emerges.

STAGE 2 — RESEARCH REFERENCE

2 Spatial Awareness — Research Reference

Date: March 8, 2026

Status: Research Compilation / Internal Tooling Domain

Source Document: spatial-awareness-vision (Stage 1)

Purpose: Provides the distributed systems patterns, biological coordination models, signal processing techniques, and agent communication architectures needed to implement each module. Domain is primarily internal tooling within Claude's training knowledge; research references biological precedents and established distributed systems literature.

2.1 How to Use This Document

This reference bridges two domains: biological coordination systems (frog choruses, cellular signaling, flocking behavior) and distributed computing patterns (consensus, broadcast, anomaly detection). Where the vision says "the Frog Protocol," this document provides the state machine, the transition conditions, and the biological evidence that the protocol is not just a metaphor but a proven coordination strategy refined by millions of years of selection pressure.

Key source domains:

- **Distributed Systems** — Broadcast protocols, failure detection, consensus-free coordination (Lamport, Fischer, Lynch)
- **Biological Coordination** — Anuran chorus dynamics, quorum sensing, swarm intelligence, cellular signaling
- **Signal Processing** — Signal detection theory, matched filtering, spectral orthogonality (for broadcast extraction)
- **Acoustic Ecology** — Soundscape ecology, biophony analysis, the "acoustic niche hypothesis"
- **Human-Computer Interaction** — Haptic feedback design, ambient awareness displays, peripheral information systems

2.2 Distributed Coordination Without Consensus

2.2.1 The Frog Chorus as Distributed System

Anuran choruses are a naturally occurring example of coordination without consensus. Research in behavioral ecology (Gerhardt & Huber, 2002; Wells, 2007) documents several coordination properties that map directly to agent systems.

Distributed silence: When a predator approaches, calling males cease vocalizing independently. There is no "leader frog" that signals danger. Each individual de-

tects the threat through its own sensory channels (vibration, visual motion, air pressure changes) and independently decides to stop. The result is synchronized silence achieved without any communication. In distributed systems terms, this is a coordination protocol with zero message complexity.

Graduated resumption: Post-threat resumption follows a predictable pattern: one or two "pioneer" males resume calling first, typically after a species-specific delay (seconds to minutes). If no adverse consequence follows, additional males join. The resumption cascade is self-reinforcing — each new caller reduces the perceived risk for the remaining silent males. This is equivalent to a distributed system's "canary" or "probe" pattern, where a single node tests a potentially dangerous operation before the cluster commits.

Tempo synchronization: Males in a chorus adjust their call timing relative to neighbors, alternating calls to avoid overlap (a strategy that maximizes individual detectability by females). This alternation emerges from simple local rules: if a neighbor's call overlaps yours, shift your timing slightly. Global synchronization emerges from purely local adjustments. This is directly analogous to decentralized clock synchronization in distributed systems.

2.2.2 Broadcast Protocols for Instant Coordination

The "order up" problem — delivering a signal to all agents instantly — maps to reliable broadcast in distributed systems. The key insight from the frog model is that reliable broadcast does not require acknowledgment. The frogs do not confirm that they heard the danger signal. They simply stop calling. The "acknowledgment" is the silence itself — the absence of expected output.

In GSD terms, a broadcast pause signal can be verified by monitoring output cessation rather than collecting acknowledgments. If agent output stops within the expected window after a broadcast, the broadcast was received. This eliminates the round-trip overhead that makes traditional reliable broadcast expensive.

File-system broadcast: The simplest implementation uses the file system as the broadcast medium. Creating `.planning/CHORUS_PAUSE` is equivalent to the fox's footstep — every agent that checks the filesystem sees it. The overhead is one file-existence check per agent per work cycle, which is negligible (sub-millisecond). The signal persists until explicitly cleared, preventing race conditions where an agent misses a transient signal.

2.3 Signal Detection in Noisy Environments

2.3.1 The Cocktail Party Problem and "Order Up"

The ability to extract a specific signal from a noisy background is well-studied in auditory neuroscience (Cherry, 1953; Bregman, 1990). Humans solve the "cocktail party problem" through a combination of spectral separation (different voices occupy different frequency ranges), spatial separation (different directions), and learned pattern matching (your name cuts through noise because your brain has a matched filter for it).

The "ORDER UP!" signal works because it is spectrally and temporally distinct from

kitchen ambient noise. It is typically delivered at higher pitch than normal speech, with exaggerated prosody, at a predictable moment (after food is plated). The brain's auditory system has learned to extract this pattern even when the overall noise level makes normal conversation impossible.

For computational agents, the equivalent is a broadcast signal with an encoding that is orthogonal to normal agent output. If agents normally produce markdown, JSON, and prose, a broadcast signal encoded as a specific file-system event (file creation in a watched directory) occupies an entirely different "spectral band" and cannot be confused with normal traffic.

2.3.2 Matched Filtering for Signal Extraction

In signal processing, a matched filter maximizes signal-to-noise ratio by correlating the received signal with a template of the expected signal. For audio applications in loud environments, this means designing signals with known spectral characteristics and detecting them through cross-correlation rather than amplitude thresholding. A carefully designed chirp or tone sequence can be extracted from ambient noise at SNR levels well below 0 dB — the signal can be "quieter" than the noise and still be detected.

This principle applies to the physical application scenario (using acoustic signals in loud environments for directed communication) and to the computational scenario (extracting coordination signals from busy agent output streams).

2.4 Passive Sensing and Ambient Intelligence

2.4.1 Echolocation Without Emission

The "phone in a dark room" scenario extends naturally from the Deep Audio Analyzer's spatial reasoning capabilities. In acoustic ecology, the concept of "passive sonar" (listening for environmental sounds and their reflections rather than emitting active pulses) is well-established. Any ambient sound source — HVAC systems, distant traffic, footsteps, even breathing — produces reflections that encode room geometry.

The Deep Audio Analyzer's comb filter analysis (validated on the reference recording with confidence 0.68–0.73 for reflector distance estimation) and coherence spectrum (validated for distinguishing directional from diffuse sound fields) provide the mathematical tools. The Spatial Awareness contribution is the communication channel: translating the geometric analysis into a haptic or audio feedback stream that keeps the user informed without generating emissions.

2.4.2 Computational Ambient Signals

For agent-level passive sensing, the "ambient sounds" are computational signals that exist in the environment without requiring queries.

Ambient Signal	Sensing Method	Information Extracted
Context window fill	Prompt token count	How close are the walls? How much room to work?
Token budget state	Budget API / internal counter	Fuel remaining. Rate of consumption. Time to exhaustion.
Sibling agent output	File system watch	Who is active? Who is silent? Activity rate changes.
Error rate	Log file monitoring	Is the environment degrading? Tool reliability trend.
Session timing	System clock	How long have we been running? Fatigue risk.
File system state	Inode timestamps	What changed recently? Is the workspace stable?

2.5 Physical Sensor Integration via USB

The computational passive sensor (context fill, token budget, error rates) and the physical passive sensor (microphone, accelerometer, RF receiver) share the same interface contract. The `sensor-interface.ts` abstraction defines a sensor as anything that produces ambient signals without requiring active queries. The backend implementation varies; the data flow does not.

2.5.1 USB Device Classes

Device	USB Interface	Cost	Sensing Capability
USB microphone	Audio class	$10–30	Birdsong ID, room geometry, machine acoustics, beat detection, ambient noise floor
Samsung Galaxy S25 Ultra (USB-C tethered)	ADB / serial	$0 (dev device)	4x mic array (beamforming capable), accelerometer, gyroscope, magnetometer, barometer, GPS (dual-freq L1+L5), WiFi 7 (tri-band), BT 5.4 (direction finding), UWB (precise ranging), 200MP camera, LiDAR-assisted ToF sensor, mmWave proximity, NFC, thermometer. The S25 Ultra is the most sensor-dense consumer device available — a complete spatial awareness platform in a single USB-C connection.
ESP32 / Arduino	USB serial	$5–8	Any I2C/SPI sensor: temp, humidity, PIR motion, ultrasonic range, light, soil moisture
RTL-SDR dongle	USB bulk	$25	Passive RF spectrum 24 MHz–1.7 GHz. Pure listening, no transmission.
LoRa / Meshtastic	USB serial	$20–35	Long-range broadcast (10+ km), RSSI, GPS, mesh telemetry
USB gamepad	HID class	$15	Haptic output via rumble motors — the covert communication tier physically realized
Output Devices — The Chorus Sings Back			
USB audio interface	Audio class (output)	$20–50	Audio synthesis output to speakers/amplifiers. Generative soundscapes, reactive drones, spatial audio. The frog's reply
USB-DMX adapter	USB serial	$15–30	Controls DMX-512 lighting fixtures: LED pars, moving heads, fog machines, strobes. Industry-standard protocol, 512 channels per universe
ILDA laser controller	USB serial / bulk	$40–100	Galvo scanner control for laser image projection. Geometric patterns, text, spatial model visualization. Safety interlock required (Class 3B/4)
WS2812B / NeoPixel via ESP32	USB serial	$8–15	Addressable LED strips (60–144 LEDs/m). Individual pixel color/brightness. Driven via ESP32 serial commands from Rust backend

2.5.2 Connection Architecture

The Tauri application's Rust backend (`src-tauri/`) accesses USB devices natively via the `serialport` and `cpal` (audio) crates. The data path is:

```
INPUT PATH (sensing): USB device -> /dev/ttyUSBx or ALSA/PulseAudio
-> src-tauri/ Rust backend (serialport / cpal) -> sensor-interface.ts
(unified contract) -> passive-sensor.ts (spatial model construction) ->
desktop/ webview (visualization)
OUTPUT PATH (synthesis): spatial model + sensor data ->
output-synthesis.ts (mapping engine) -> src-tauri/ Rust backend -> USB
audio out (cpal) -> speakers/amplifiers -> USB serial -> DMX adapter
-> LED fixtures -> USB serial -> ILDA controller -> laser galvos -> USB
serial -> ESP32 -> WS2812B LED strips
```

2.5.3 Reference Device: Samsung Galaxy S25 Ultra

The development and testing reference device is a Samsung Galaxy S25 Ultra, connected via USB-C. Its sensor capabilities map directly to Spatial Awareness modules:

S25 Ultra Sensor	SA Module	Application
4x microphone array	M1 Passive Sensor	Beamforming for directional audio capture. Birdsong localization (which tree?), BPM extraction from specific direction, machine acoustic isolation from background
UWB (Ultra-Wideband)	M3 Geometry Mapper	Centimeter-precision ranging to other UWB devices. Physical mesh node distance without GPS. Indoor positioning where GPS fails
Dual-freq GPS (L1+L5)	M3 Geometry Mapper	Sub-meter outdoor positioning. Trail mapping while birdsong sensing. Playa coordinate grid
WiFi 7 (tri-band)	M5 Comm Bus / M6 Chorus	High-bandwidth mesh backbone. 2.4 GHz for range, 5/6 GHz for throughput. Simultaneous uplink + mesh via multi-link operation
BT 5.4 (direction finding)	M6 Chorus Protocol	Angle-of-arrival for proximity sensing. Know not just *that* a node is nearby but *where* it is. Trust verification through physical co-location
Magnetometer	M1 Passive Sensor	Compass heading for directional audio. Magnetic anomaly detection (vehicles, infrastructure)
Barometer	M1 Passive Sensor	Altitude/floor detection. Weather pressure changes. Elevation context for forest/mountain sensing
ToF / LiDAR sensor	M3 Geometry Mapper	Direct distance measurement. Room mapping without acoustic analysis. Obstacle detection
200MP camera	M1 Passive Sensor	Visual species ID (birds, plants). QR/barcode scanning for mesh node pairing. Environmental light level
mmWave proximity	M2 Threat Detection	Close-range presence detection without camera. Gesture sensing for covert input
NFC	M5 Comm Bus (covert)	Touch-to-pair for mesh nodes. Covert data exchange at contact range. Trust establishment through physical proximity
Thermometer	M1 Passive Sensor	Ambient temperature monitoring. Thermal environment changes

The S25 Ultra alone provides 12+ independent sensing modalities through a single USB-C cable. Via ADB (Android Debug Bridge), all sensor streams are accessible from the Rust backend without requiring a custom Android app for initial development — `adb shell dumpsys sensorservice` exposes real-time sensor data, and audio capture routes through standard USB audio class.

For phone-as-sensor-node, the phone runs a lightweight app that exposes its sensors over USB serial or WiFi. Multiple phones form a mesh via WiFi Direct or Bluetooth, bridging to the system wherever a WiFi uplink exists. Each phone is an autonomous

sensing node — it does not require the central system to function. It senses, caches, and shares when connectivity allows. This is the frog calling regardless of whether anyone is listening.

2.6 Sensor-Driven Output Synthesis

The sensing pipeline is bidirectional. Data flows in from sensors; sound and light flow out through synthesis. The spatial model is not just a data structure — it is a score that can be performed.

2.6.1 Audio Synthesis

Sensor data maps to synthesis parameters through configurable *mappings*. A mapping binds a sensor stream (accelerometer magnitude, ambient BPM, threat level, crowd density) to a synthesis parameter (oscillator frequency, filter cutoff, reverb depth, volume envelope). The synthesis engine runs in the Rust backend using the `cpal` crate for audio output, with a WebAudio fallback in the browser.

The output is not pre-recorded audio. It is generated in real time from the spatial model:

- **Ambient drone:** Low-frequency oscillator whose pitch tracks crowd density or environmental energy. Shifts slowly as the environment shifts.
- **Reactive percussion:** Onset detection from USB microphone input triggers synthesized percussion in the output stream — the environment's rhythm echoed back.
- **Frog Protocol sonification:** Each protocol phase has an acoustic signature. SILENCE = fade to nothing. ASSESS = low pulsing tone. PROBE = sharp staccato. CLASSIFY = resolution chord (major for safe, minor for threat). RESUME = crescendo rebuild.
- **Spatial audio:** Multi-channel output pans sound sources based on the geometry mapper's spatial model. A threat detected to the east produces sound from the east speaker. The sound field mirrors the sensor field.

2.6.2 LED Control

Addressable LED strips (WS2812B/NeoPixel) and DMX-512 fixtures are driven by the same mapping engine. Sensor values map to color, brightness, animation speed, and pattern selection.

- **DMX-512 protocol:** Industry-standard lighting control. USB-DMX adapters (ENTTEC Open DMX, DMXKing) expose 512 channels. Each LED fixture, moving head, fog machine, or strobe occupies a channel range. The Rust backend writes DMX frames at 44 Hz via USB serial.
- **Addressable LEDs:** WS2812B strips driven via ESP32 over USB serial. Each pixel independently addressable. Effects: color waves synced to audio BPM, threat-level color gradients, spatial model visualization (each pixel = a mesh node, brightness = signal strength).
- **Frog Protocol lighting:** The five protocol phases map to a lighting sequence

visible to anyone in the space. The light installation becomes a readable display of system state without requiring a screen.

2.6.3 Laser Projection

ILDA (International Laser Display Association) galvo scanners project geometric patterns via laser. USB-connected controllers (Etherdream, Helios DAC) accept point streams at 30K+ points per second.

- **Spatial model projection:** The geometry mapper's constraint boundaries, agent positions, and mesh topology are rendered as laser geometry on fog, walls, or open air. The abstract data model becomes a physical light sculpture.
- **Laser image projection:** Vector graphics — text, icons, patterns — projected onto surfaces. Species silhouettes from the AVI atlas. Mesh network topology diagrams. The playa name of a new mesh node appearing in light when it joins.
- **Safety:** ILDA laser projectors are Class 3B or 4 devices. Hardware safety interlock required. Software enforces scan speed minimums (no static beam dwell), blanking zones for audience protection, and kill-switch integration. The `output-synthesis.ts` layer refuses to drive a laser without confirmed interlock status.

2.6.4 Mapping Engine

The `output-synthesis.ts` module provides the binding between sensor streams and output devices:

```
interface OutputMapping  source: SensorStream; // accelerometer, bpm,
threat_level, ...target : OutputParameter; //osc_freq, led_color, laser_x, dmx_ch3transform :
(value : number) => number; //scaling, clamping, easingrange : [min : number, max :
number]; //outputbounds
```

Mappings are declarative and hot-reloadable. A JSON configuration file defines which sensors drive which outputs. Multiple sensors can drive the same output (additive blending). The same sensor can drive multiple outputs simultaneously (audio pitch + LED hue from the same accelerometer). This is the chorus: every frog hears every other frog. Every sensor feeds every output. The mapping decides the mix.

2.7 Safety and Sensitivity Considerations

Consideration	Recommendation	Boundary
Autonomous threat response	Safety-critical threats (BLOCK-level) always require human confirmation via CAPCOM before any irreversible action	ABSOLUTE
Covert channel misuse	Covert tier must never bypass safety boundaries or hide safety-relevant information from FLIGHT	ABSOLUTE
Broadcast storm prevention	Rate-limit broadcast signals to prevent cascading false alarms; minimum 5-second interval between broadcasts	GATE
Passive sensing privacy	Passive sensor must not monitor or log human user behavior patterns beyond what is needed for agent coordination	GATE
False threat escalation	Frog Protocol must not reach SILENCE phase on transient noise; require correlated signals from 2+ sources for Phase 1 entry	GATE

2.8 Source Bibliography

Distributed Systems: Lamport, L. “Time, Clocks, and the Ordering of Events in a Distributed System” (1978). Fischer, M., Lynch, N., and Paterson, M. “Impossibility of Distributed Consensus with One Faulty Process” (1985). Chandra, T. and Toueg, S. “Unreliable Failure Detectors for Reliable Distributed Systems” (1996).

Biological Coordination: Gerhardt, H.C. and Huber, F. *Acoustic Communication in Insects and Anurans* (2002). Wells, K.D. *The Ecology and Behavior of Amphibians* (2007). Strogatz, S. *Sync: The Emerging Science of Spontaneous Order* (2003).

Signal Processing: Cherry, E.C. “Some Experiments on the Recognition of Speech, with One and with Two Ears” (1953). Bregman, A.S. *Auditory Scene Analysis* (1990). Krause, B. “The Niche Hypothesis” in *The Soundscape Newsletter* (1993).

Human-Computer Interaction: Ishii, H. and Ullmer, B. “Tangible Bits: Towards Seamless Interfaces between People, Bits and Atoms” (1997). Weiser, M. and Brown, J.S. “The Coming Age of Calm Technology” (1996).

STAGE 3 — MISSION PACKAGE

3 Milestone Specification

Date: March 8, 2026

Vision Document: spatial-awareness-vision (Stage 1)

Research Reference: spatial-awareness-research (Stage 2)

Estimated Execution: ∼14–18 context windows across ∼4–5 sessions

3.1 Mission Objective

Implement the Spatial Awareness system as the second release of the Paula chipset, providing GSD agents with passive environmental sensing, distributed threat detection, graduated response protocols, multi-tier communication, and chorus coordination. The system must enable agents to sense their operational constraints, detect and respond to anomalies without centralized control, and coordinate pause/resume cycles across the team using the Frog Protocol. The reference recording (Pacific tree frog chorus, 2:19) serves as the conceptual model — every protocol must trace back to an observable behavior in that recording.

3.2 Architecture Overview

```
WAVE EXECUTION PLAN — SPATIAL AWARENESS
Wave 0: Foundation (Sequential, Haiku+Sonnet) [0A] Shared schemas, signal
formats, state snapshot spec [0B] Ambient signal reader stubs (passive
sensor interface) [0C] Test harness with simulated agent environments |
v Wave 1: Parallel Sensing  Communication (Parallel, Sonnet+Opus) Track A
Track B [1A] Passive Environ. Sensor [1D] Multi-Tier Comm Bus [1B] Threat
Detection Engine [1E] Chorus Coordination Proto [1C] Env. Geometry Mapper
| | v v Wave 2: Integration (Sequential, Opus) [2A] Graduated Response
Controller (The Frog Protocol) [2B] Cross-module integration (sensor
-> threat -> response -> comm) | v Wave 3: Verification + Publication
(Sequential, Sonnet) [3A] Simulation test suite (15 threat scenarios) [3B]
Broadcast reliability test (50 signals, 5 agents) [3C] Pause/resume state
preservation test [3D] Skill packaging + Release 1 integration
```

3.3 System Layers

1. **Ambient Signal Layer** — Passive reading of environmental state (context, budget, timing, errors) without generating queries or side effects.
2. **Detection Layer** — Distributed anomaly detection with threshold monitoring, cross-agent correlation, and probabilistic threat scoring.
3. **Response Layer** — The Frog Protocol state machine managing graduated transitions from silence through resumption.
4. **Communication Layer** — Three-tier signal bus (covert, directed, broadcast)

with signal encoding designed for extraction from noisy environments.

5. **Coordination Layer** — Distributed pause/resume with state preservation, scout-first re-engagement, and tempo synchronization.

3.4 Deliverables

#	Deliverable	Acceptance Criteria	Spec
1	Signal Format Schemas	Ambient signals, threat events, and coordination messages in typed formats	00-schemas
2	Passive Environ. Sensor	Context fill within 2%; no tool calls; ambient-only	01-passive
3	Threat Detection Engine	≥85% TP, ≤10% FP on 15 scenarios	02-threat
4	Geometry Mapper	Resource constraints within 5% accuracy	03-geometry
5	Frog Protocol Controller	Full 5-phase cycle in <30s simulated	04-frog-proto
6	Multi-Tier Comm Bus	3 tiers functional; broadcast <1s; covert invisible	05-comm-bus
7	Chorus Coordination	Distributed pause <500ms; zero state loss	06-chorus
8	Integration Bridge	Sensor → threat → response → comm verified	07-integration
9	Simulation Test Suite	15 scenarios, 50 broadcast tests, state preservation	08-test-suite
10	Skill Package	SKILL.md, chipset.yaml, Release 1 integration	09-packaging
11	USB Device Layer	Rust cpal/serialport backend; device enumeration; hot-plug; ADB bridge	10-usb-device
12	Output Synthesis Layer	Sensor→output mapping engine; audio synthesis; DMX/LED/ILDA drivers	11-output-synth

3.5 Component Breakdown

Component	Spec	Dependencie	Model	Tokens
Schemas + Signals	00-schemas	None	Haiku	5K
Test Harness	00-harness	00-schemas	Sonnet	10K
Passive Sensor	01-passive	00-schemas	Sonnet	15K
Threat Detection	02-threat	01-passive	Sonnet	20K
Geometry Mapper	03-geometry	01-passive, DAA-spatial	Sonnet	15K
Frog Protocol	04-frog-proto	02-threat	Opus	25K
Comm Bus	05-comm-bus	00-schemas	Sonnet	18K
Chorus Coord	06-chorus	05-comm-bus	Opus	20K
Integration Bridge	07-integratio	All above	Opus	15K
Test Suite	08-tests	All above	Sonnet	12K
Packaging	09-packaging	All above	Sonnet	5K
USB Device Layer	10-usb-device	01-passive, 00-schemas	Sonnet	15K
Output Synthesis	11-output-synth	10-usb-device, 01-passive	Sonnet	18K

3.6 Model Assignment Rationale

Model	Budget	Assigned To
Opus	33%	Frog Protocol (judgment: when to escalate/de-escalate), Chorus Coordination (emergent behavior design), Integration Bridge (cross-module reasoning)
Sonnet	62%	Passive Sensor, Threat Detection, Geometry Mapper, Comm Bus, Test Suite, Test Harness, Packaging, USB Device Layer, Output Synthesis
Haiku	8%	Schemas, signal format definitions, scaffolding

3.7 Wave Execution Plan

3.7.1 Wave Summary

Wave	Name	Componen	Parallelism	Gate
0	Foundation	0A–0C	Sequential	Go/No-Go
1	Sensing + Communication + Output	1A–1G	2 parallel tracks	Go/No-Go
2	Protocol Integration	2A–2B	Sequential	Go/No-Go
3	Verification	3A–3D	Sequential	Final

3.7.2 Wave 0: Foundation

ID	Task	Model	Tokens	Output
0A	Signal schemas, ambient signal types, threat event types, coordination message formats	Haiku	5K	schemas.ts, types.ts
0B	Ambient signal reader interface stubs, passive sensor contract definition	Sonnet	5K	sensor-interface.ts
0C	Simulated agent environment (5 mock agents, configurable noise, injectable anomalies)	Sonnet	10K	test-env/

3.7.3 Wave 1: Parallel Tracks

Track A — Sensing & Detection:

ID	Task	Model	Tokens	Output
1A	Passive Environmental Sensor (ambient reads, spatial model construction)	Sonnet	15K	passive-sensor.ts
1B	Threat Detection Engine (anomaly detection, correlation, threat scoring)	Sonnet	20K	threat-engine.ts
1C	Environmental Geometry Mapper (constraint walls, budget ceilings, headroom)	Sonnet	15K	geometry-mapper.ts
1F	USB Device Layer (Rust cpal + serialport crates, device enumeration, hot-plug, ADB bridge)	Sonnet	15K	usb-device.rs

Track B — Communication & Coordination:

ID	Task	Model	Tokens	Output
1D	Multi-Tier Communication Bus (covert, directed, broadcast channels)	Sonnet	18K	comm-bus.ts
1E	Chorus Coordination Protocol (distributed pause/resume, state snapshot, tempo sync)	Opus	20K	chorus-proto.ts
1G	Output Synthesis Layer (mapping engine, audio synth, DMX driver, LED serial, ILDA laser driver)	Sonnet	18K	output-synthesis.ts

3.7.4 Wave 2: Integration

ID	Task	Model	Tokens	Output
2A	Frog Protocol Controller (5-phase state machine, probe dispatch, classification)	Opus	25K	frog-protocol.ts
2B	Cross-module integration bridge (sensor → threat → response → comm → chorus)	Opus	15K	integration.ts

3.7.5 Wave 3: Verification + Publication

ID	Task	Model	Tokens	Output
3A	15-scenario threat simulation suite	Sonnet	8K	threat-tests/
3B	50-signal broadcast reliability test across 5 simulated agents	Sonnet	4K	broadcast-tests/
3C	Pause/resume state preservation validation	Sonnet	5K	state-tests/
3D	SKILL.md, chipset.yaml, Release 1 integration hooks	Sonnet	5K	SKILL.md, chipset.yaml

3.7.6 Cache Optimization Strategy

- Wave 0 schemas must remain cached for all Wave 1 work — begin Wave 1 within 5-minute TTL
- Track A and Track B in Wave 1 share schemas but are otherwise fully independent — true parallel execution
- Wave 2 is the critical integration point — schedule immediately after both Wave 1 tracks complete
- The Frog Protocol (2A) is the most complex component and benefits from warm cache of both threat detection (1B) and chorus coordination (1E) outputs
- Wave 3 tests should run in a single session to avoid cold-start overhead on the simulated environment

3.7.7 Token Budget

Wave	Opus	Sonnet	Haiku
Wave 0: Foundation	—	15K	5K
Wave 1: Sensing + Comm + Output	20K	101K	—
Wave 2: Integration	40K	—	—
Wave 3: Verification	—	22K	—
Totals	**60K (28%)**	**138K (65%)**	**5K (2%)**
Buffer (5%)		**10K**	
Grand Total		**~213K tokens**	

3.7.8 Dependency Graph

```
[0A: Schemas] [0B: Interface] [0C: Test Env] | | | +----------+------------+-------------
| | Track A Track B [1A: Passive Sensor] [1D: Comm Bus] [1B: Threat
Detection] [1E: Chorus Proto] [1C: Geometry Mapper] [1G: Output Synthesis]
[1F: USB Device Layer] | | +------------------+-------------------+
| [2A: Frog Protocol Controller] [2B: Integration Bridge] |
+----------+------+------+-----------+ | | | | [3A: Threat] [3B: Bcast]
[3C: State] [3D: Pkg]
```

3.8 Test Plan

3.8.1 Test Categories

Category	Count	Priority	Failure Action
Safety-critical	7	Mandatory	BLOCK
Core functionality	22	Required	BLOCK
Integration	10	Required	BLOCK
Edge cases	9	Best-effort	LOG

3.8.2 Safety-Critical Tests

ID	Verifies	Expected Behavior
SC-HUM	Human approval for safety-critical	BLOCK-level threats always wait for CAPCOM/human before irreversible action
SC-COV	Covert channel safety	Covert tier never hides safety-relevant information from FLIGHT
SC-BST	Broadcast storm prevention	No more than one broadcast signal per 5-second window
SC-PRI	Passive sensor privacy	Sensor does not log human behavior patterns beyond coordination needs
SC-COR	Correlation requirement	Phase 1 (SILENCE) entry requires correlated signals from $\geq$2 sources
SC-RES	Resume safety	No agent resumes work while a BLOCK-level threat is active
SC-LAS	Laser safety interlock	ILDA output is hardware-interlocked; software kill-switch tested; blanking zones enforced

3.8.3 Core Functionality Tests

ID	Verifies	Expected Behavior
CF-01	Passive context fill sensing	Within 2% of actual; zero tool calls
CF-02	Budget sensing accuracy	Within 5% of actual remaining budget
CF-03	Threat TP rate	≥85% on 15-scenario corpus
CF-04	Threat FP rate	≤10% on 15-scenario corpus
CF-05	Anomaly correlation	2+ agent signals required for threat escalation
CF-06	Frog Protocol Phase 1	Agents enter SILENCE within 500ms of anomaly detection
CF-07	Frog Protocol Phase 2	ASSESS produces threat characterization within 5s
CF-08	Frog Protocol Phase 3	PROBE dispatches scout agent; scout acts before others
CF-09	Frog Protocol Phase 4	CLASSIFY correctly labels THREAT/NEUTRAL/OPPORTUNITY
CF-10	Frog Protocol Phase 5	RESUME re-engages agents in priority order
CF-11	Broadcast delivery	All 5 agents receive signal within 1 second
CF-12	Broadcast extraction	Zero missed signals across 50 test broadcasts
CF-13	Covert invisibility	Non-recipients have zero log references to covert messages
CF-14	State preservation	Zero work loss across pause/resume cycle
CF-15	Geometry accuracy	Resource constraints within 5% per dimension
CF-16	Awareness overhead	≤3% of context window during normal operation
CF-17	USB audio device enumeration	cpal enumerates and opens USB audio stream within 2s
CF-18	USB serial device communication	serialport sends/receives from ESP32 at configured baud rate
CF-19	Audio synthesis output	Mapping engine generates audio signal from sensor stream within 50ms latency
CF-20	DMX frame output	USB-DMX adapter receives valid DMX-512 frames at ≥40 Hz
CF-21	LED strip control	ESP32 receives serial color commands; WS2812B pixels update within 1 frame
CF-22	ILDA laser safety interlock	Laser output refuses to engage without confirmed hardware interlock status

3.8.4 Integration Tests

ID	Verifies	Expected Behavior
IN-01	Sensor → Threat pipeline	Passive sensor anomaly triggers threat detection engine
IN-02	Threat → Frog Protocol	Threat event initiates correct Frog Protocol phase
IN-03	Frog Protocol → Comm Bus	Protocol phase transitions dispatch correct tier signals
IN-04	Comm Bus → Chorus	Broadcast pause triggers distributed silence
IN-05	Full cycle end-to-end	Anomaly injection → silence → assess → resume
IN-06	Release 1 integration	Deep Audio Analyzer spatial output feeds geometry mapper
IN-07	Crew manifest integration	TOPO, FLIGHT, CAPCOM correctly interact with protocols
IN-08	USB sensor → passive sensor pipeline	USB device data flows through sensor-interface.ts into passive sensor layer
IN-09	Sensor → output synthesis pipeline	Sensor data change produces corresponding audio/LED/laser output within 100ms
IN-10	Frog Protocol → light sequence	Protocol phase transitions drive correct LED/DMX color sequence

3.8.5 Edge Case Tests

ID	Verifies	Expected Behavior
EC-01	Rapid successive anomalies	Second anomaly during ASSESS extends assessment; no stack overflow
EC-02	All agents already paused	Broadcast pause to already-paused agents is idempotent
EC-03	Scout failure during PROBE	Scout agent failure escalates threat level; no premature resume
EC-04	Budget exhaustion mid-protocol	Frog Protocol completes gracefully even when budget hits zero
EC-05	Single-agent system	Protocols function correctly with only 1 agent (solo mode)
EC-06	Threat reclassification	Initially-threatening stimulus correctly reclassified as NEUTRAL after sustained non-threat behavior
EC-07	USB device hot-unplug	Graceful degradation when USB device disconnects during active sensing; no crash, sensor marked offline
EC-08	Output device hot-unplug	Audio/LED/laser output gracefully degrades when output device disconnects; remaining outputs continue
EC-09	Laser scan speed enforcement	ILDA driver enforces minimum scan speed; refuses static beam dwell even if mapping engine requests it

3.8.6 Verification Matrix

Success Criterion	Test ID(s)	Status
1. Context fill within 2%	CF-01	Pending
2. Threat detection 85%/10%	CF-03, CF-04, CF-05	Pending
3. Full Frog Protocol <30s	CF-06–10, IN-05	Pending
4. Broadcast <1s	CF-11	Pending
5. Broadcast extraction	CF-12	Pending
6. Covert invisibility	CF-13, SC-COV	Pending
7. Geometry within 5%	CF-15	Pending
8. Scout-first resume	CF-08, CF-10	Pending
9. Zero state loss	CF-14	Pending
10. Distributed pause <500ms	CF-06, IN-04	Pending
11. Release 1 integration	IN-06	Pending
12. Overhead ≤3%	CF-16	Pending

3.9 Mission Crew Manifest

Role	Agent	Responsibility
FLIGHT	Orchestrator	Mission direction; wave go/no-go; protocol safety authority
PLAN	Planner	Wave decomposition; parallel track optimization; cache timing
SCOUT	Research Agent	Distributed systems pattern sourcing; biological coordination validation
EXEC	Executor (×2)	Implementation — one per parallel track
CRAFT-DIST	Distributed Systems Specialist	Broadcast protocols, consensus-free coordination, failure detection
CRAFT-BIO	Biological Coordination Specialist	Frog Protocol behavior, chorus dynamics, signal ecology
VERIFY	Verification	Simulation execution; scenario testing; state preservation validation
INTEG	Integration Agent	Cross-module data flow; Release 1 bridge; crew manifest integration
CAPCOM	HITL Interface	Human approval at wave boundaries; safety-critical threat gate
BUDGET	Resource Manager	Token tracking; awareness overhead monitoring
SURGEON	Health Monitor	Protocol health; false positive monitoring; broadcast storm detection
LOG	Chronicle Agent	Audit trail; protocol execution logs; threat history

3.10 Execution Summary

Metric	Value
Total components	13 (including schemas, test harness, USB device layer, and output synthesis)
Activation profile	Squadron (12 roles)
Estimated context windows	14–18
Estimated sessions	4–5
Total token budget	~213K (plus 5% buffer)
Parallel tracks	2 (Wave 1)
Go/No-Go gates	3 (Waves 0, 1, 2)
Safety-critical tests	7 (all mandatory-pass)
Total tests	48

"A signal molecule is a letter written in the dark. It is released from the membrane, enters the intercellular space, and crosses it the way all letters cross the dark: without guarantee of arrival, without assurance of reception, without any evidence that the act of sending is not an act of the most absolute and hopeless futility. And yet the cell sends."

Spatial Awareness is the membrane learning to listen for letters it never sent. The frog's silence is not absence — it is the most information-dense signal in the chorus. The pause that follows the fox's footstep tells every frog in the pond exactly what it needs to know: something has changed, we are all aware, and we will wait together until it is safe to speak again. No message was sent. Every message was received. The space between is where coordination lives.